

Fourier

A Modern Programming Language for
Machine Learning and Graphics

Language Specification v0.1

Mario Michael Heinrich

Founder of easy-job.ai

github.com/Engineer1080

Draft: January 2026

Table of Contents

1.	Introduction	3
2.	Design Philosophy	3
3.	Core Features	4
4.	Syntax Overview	5
5.	Execution Modifiers	6
6.	Type System	7
7.	Built-in Algorithms	8
8.	Parallelization	9
9.	GPU/TPU Integration	10
10.	Standard Library	11
11.	Use Cases	12
12.	Implementation Roadmap	13

1. Introduction

Fourier is a modern, high-performance programming language designed specifically for compute-intensive workloads in machine learning and graphics rendering. Named after Joseph Fourier, whose mathematical transformations are fundamental to both signal processing and modern graphics, the language embodies the principle of efficient transformation of data through parallel computation.

Unlike general-purpose languages retrofitted with parallel computing capabilities, Fourier is built from the ground up with GPU/TPU acceleration and automatic parallelization as core features. This allows developers to write intuitive, readable code that automatically leverages modern hardware architectures without manual optimization.

2. Design Philosophy

2.1 Core Principles

- **Parallel by Default:** Operations are automatically parallelized unless explicitly marked sequential
- **Hardware Agnostic:** Write once, run efficiently on CPU, GPU, or TPU
- **Readable Imperative Syntax:** Familiar syntax for easier adoption, functional semantics for safety
- **Smart Compilation:** Compiler makes intelligent decisions about execution strategy
- **Explicit Control When Needed:** Developers can override defaults with simple modifiers

2.2 Target Domains

Fourier is optimized for two primary domains that share common computational characteristics: machine learning (training and inference) and graphics rendering (real-time and offline). Both require massive parallelism, matrix operations, and efficient GPU utilization.

3. Core Features

Automatic Parallelization

All operations are parallelized by default across available compute resources

Execution Modifiers

Simple `.gpu()`, `.cpu()`, and `.single()` modifiers for explicit control

Built-in Algorithms

Comprehensive standard library of optimized algorithms for common tasks

Work-Stealing Scheduler

Dynamic load balancing ensures optimal resource utilization

Native Vector/Matrix Types

First-class support for mathematical data structures

Cross-Platform

Runs on Windows, macOS, and Linux

LLVM/MLIR Backend

Modern compiler infrastructure for optimal code generation

Interoperability

WebAssembly support and FFI for existing C/C++ libraries

4. Syntax Overview

4.1 Variables and Basic Types

```
// Variable declaration

x = 42

name = "Fourier"

pi = 3.14159


// Type inference is automatic

numbers = [1, 2, 3, 4, 5]

matrix = [[1, 2], [3, 4]]
```

4.2 Functions

```
// Function definition

fn add(a, b) {

  return a + b

}


// Lambda functions

square = x => x * x

multiply = (a, b) => a * b
```

4.3 Control Flow

```
// If-else

if x > 0 {

  print("positive")

} else {

  print("non-positive")

}
```

```
// Loops
```

```
for item in collection {
```

```
  process(item)
```

```
}
```

```
while condition {
```

```
  do_something()
```

```
}
```

5. Execution Modifiers

Fourier provides simple execution modifiers that allow developers to control where and how their code executes. These modifiers are applied at the end of an operation chain to avoid inefficient data transfers between compute units.

5.1 Available Modifiers

```
// Default: Compiler decides optimal execution
```

```
result = data.map(process)
```

```
// Force GPU execution
```

```
result = data.map(heavy_computation).gpu()
```

```
// Force CPU execution (all cores)
```

```
result = data.map(io_operation).cpu()
```

```
// Single-threaded execution
```

```
result = data.map(sequential_task).single()
```

```
// Chaining operations before modifier
```

```
result = data
```

```
.filter(is_valid)
```

```
.map(transform)
```

```
.reduce(combine)
```

```
.gpu() // Entire chain runs on GPU
```

5.2 Modifier Rules

- Modifiers can only appear at the **end** of an operation chain
- Only **one modifier** per chain is allowed
- Attempting to use multiple modifiers (e.g., .gpu().cpu()) results in a **compile error**
- This design prevents inefficient GPU↔CPU data transfers

6. Type System

Fourier uses a static type system with type inference. Types are checked at compile-time to prevent runtime errors, but developers rarely need to write explicit type annotations.

6.1 Primitive Types

```
// Numeric types
int8, int16, int32, int64
uint8, uint16, uint32, uint64
float16, float32, float64

// Boolean
bool

// String
string

// Precision control for GPU operations
weights = initialize_matrix().float32()
colors = calculate_pixel().float16() // Graphics can use lower precision
```

6.2 Collection Types

```
// Arrays (homogeneous)
numbers: [int32] = [1, 2, 3, 4]

// Vectors (mathematical)
position: vec3 = vec3(1.0, 2.0, 3.0)
color: vec4 = vec4(1.0, 0.5, 0.0, 1.0)

// Matrices
transform: mat4 = identity_matrix()
weights: mat3x4 = random_matrix(3, 4)
```

6.3 Memory Location Attributes

Every variable and object has a built-in `.address` attribute that exposes its memory location. This allows for explicit memory management when needed while maintaining safety.

```
// Check if two variables point to same object
```

```
obj1 = create_object()
```

```
obj2 = obj1
```

```
if obj1.address == obj2.address {
```

```
    print("Same object")
```

```
}
```

```
// The .address attribute is read-only for safety
```

```
// This prevents arbitrary memory access
```

7. Built-in Algorithms

Fourier provides a comprehensive standard library of optimized algorithms. These are implemented with hardware-specific optimizations and automatically select the best implementation based on data size and available hardware.

7.1 Algorithm Categories

Category	Algorithms
Sorting & Search	quicksort, mergesort, radixsort, binary_search, nth_element
Machine Learning	convolution, pooling, softmax, relu, sigmoid, batch_norm, attention, backprop
Matrix Operations	matmul, dot_product, transpose, inverse, eigen_decomposition, svd, fft, ifft
Graphics	ray_trace, rasterize, blur, edge_detect, transform, lighting_phong, lighting_pbr
Graph Algorithms	dijkstra, a_star, bfs, dfs, minimum_spanning_tree

7.2 Algorithm Usage Examples

```
// Sorting automatically chooses best algorithm
sorted_data = data.sort()

// Or specify algorithm explicitly
sorted_data = data.sort(algorithm: quicksort)

// With constraints
sorted_data = data.sort(stable: true) // Must be stable
sorted_data = data.sort(inplace: true) // In-place sorting

// ML operations
output = input.convolution(kernel)
activated = logits.softmax()
normalized = features.batch_norm()

// Graphics operations
blurred = image.blur(radius: 5)
lit_scene = scene.lighting_pbr(lights, camera)
```

8. Parallelization

Fourier's parallelization system is based on automatic detection of data dependencies and a sophisticated work-stealing scheduler. Operations are parallelized by default, with the compiler and runtime working together to achieve optimal performance.

8.1 Automatic Parallelization

```
// These operations run in parallel automatically

results = large_dataset.map(expensive_function)

// Nested parallelization handled intelligently

processed = images.map(image => {
  return image.pixels.map(process_pixel)
})

// Work-stealing ensures balanced load

// Fast cores steal tasks from slower ones
```

8.2 Work-Stealing Scheduler

The runtime uses a work-stealing scheduler where idle threads steal tasks from busy threads. This ensures optimal load balancing even when tasks have varying execution times or when dealing with nested parallelism.

- **Dynamic Load Balancing:** Tasks are distributed at runtime based on actual execution patterns
- **Cache-Friendly:** Work stealing minimizes cache misses by preferring local work
- **Nested Parallelism:** Handles multiple levels of parallelism without thread explosion
- **Low Overhead:** Efficient task queues minimize scheduling overhead

8.3 When to Use `.single()`

While parallelization is the default, there are legitimate cases for single-threaded execution:

- **Debugging:** Sequential execution makes debugging easier
- **Order Dependencies:** When operations must execute in specific order
- **Small Data:** Parallelization overhead exceeds benefits
- **Resource Control:** When other tasks are using available cores

```
// Debug mode: sequential execution
```

```
result = data.map(buggy_function).single()
```

```
// Stateful reduction requiring order
```

```
final_state = events.fold(initial, update).single()
```

9. GPU/TPU Integration

Fourier treats GPUs and TPUs as first-class compute resources. The compiler automatically generates optimized kernels for supported operations, and the runtime manages data transfer and synchronization.

9.1 Automatic GPU Utilization

The compiler analyzes code to determine which operations benefit from GPU acceleration. Operations involving large arrays, matrix operations, or data-parallel computations are automatically candidates for GPU execution.

```
// Compiler may automatically use GPU for large operations
```

```
huge_matrix = random_matrix(10000, 10000)
```

```
result = huge_matrix.matmul(another_matrix)
```

```
// Or force GPU execution
```

```
result = huge_matrix.matmul(another_matrix).gpu()
```

```
// CPU for small operations (avoid transfer overhead)
```

```
tiny_calc = small_vector.normalize().cpu()
```

9.2 Memory Management

Fourier's runtime automatically manages data transfer between CPU and GPU memory. Data is kept on the GPU between operations when possible to minimize expensive transfers.

```
// This entire chain runs on GPU - no intermediate transfers
```

```
result = data
```

```
.normalize()
```

```
.convolution(kernel)
```

```
.relu()
```

```
.pooling()
```

```
.gpu()
```

```
// Data stays on GPU until final result is needed
```

9.3 Precision Control

Different workloads require different precision levels. ML training typically needs float32, while graphics rendering can often use float16 for better performance.

```
// ML training: higher precision
weights = initialize_weights().float32()
gradients = compute_gradients(weights).float32()

// Graphics rendering: lower precision acceptable
fragment_color = shade_pixel(position, normal).float16()

// Mixed precision training
loss = forward_pass(inputs.float16()).float32()
gradients = backward_pass(loss).float32()
```

10. Standard Library

Fourier's standard library is organized into modules focused on specific domains. All modules are designed with parallelization and GPU acceleration in mind.

Module	Description
fourier.ml	Machine learning operations (layers, optimizers, losses)
fourier.graphics	Graphics primitives, shaders, rendering pipelines
fourier.linalg	Linear algebra operations (matrices, vectors, decompositions)
fourier.signal	Signal processing (FFT, filters, convolutions)
fourier.image	Image processing operations
fourier.data	Data structures and algorithms
fourier.io	Input/output operations
fourier.random	Random number generation (CPU and GPU)

10.1 Example Usage

```
import fourier.ml as ml

import fourier.graphics as gfx


// Build a neural network

model = ml.Sequential([
  ml.Conv2D(32, kernel_size: 3),
  ml.ReLU(),
  ml.MaxPool2D(pool_size: 2),
  ml.Dense(10)
])


// Train on GPU

trained_model = model.train(data, epochs: 10).gpu()


// Render a 3D scene

scene = gfx.Scene()

scene.add(gfx.Mesh(vertices, indices))
```



```
frame = scene.render(camera).gpu()
```

11. Use Cases

Fourier is designed for compute-intensive workloads that benefit from massive parallelization. Here are the primary use cases:

11.1 Machine Learning

- **Neural Network Training:** Train deep learning models efficiently
- **Inference at Scale:** Deploy models for high-throughput inference
- **Computer Vision:** Process images and video in real-time
- **Natural Language Processing:** Train and run transformer models
- **Reinforcement Learning:** Simulate environments and train agents

11.2 Graphics and Gaming

- **Real-time Rendering:** Game engines with 60+ FPS requirements
- **Ray Tracing:** Physically-based rendering for realistic graphics
- **Physics Simulation:** Real-time physics for games and simulations
- **Video Processing:** Encoding, decoding, and effects
- **Procedural Generation:** Generate textures, terrains, and content

11.3 Scientific Computing

- **Numerical Simulations:** Climate models, fluid dynamics, molecular dynamics
- **Signal Processing:** Audio processing, radar, communications
- **Financial Modeling:** Monte Carlo simulations, risk analysis
- **Data Analysis:** Large-scale statistical analysis

- **Optimization Problems:** Solve complex optimization tasks

12. Implementation Roadmap

The development of Fourier will proceed in phases, with each phase building on the previous one. This roadmap outlines the planned development trajectory.

Phase 1: Specification & Design (Current)

- Finalize language syntax and semantics
- Define core type system
- Document standard library APIs
- Review relevant research papers (MLIR, Triton, etc.)

Phase 2: Proof-of-Concept Interpreter

- Implement basic interpreter in Python
- Test syntax and language feel
- Validate design decisions
- Build simple example programs

Phase 3: Frontend Development

- Implement lexer and parser
- Build type checker and semantic analyzer
- Create error reporting system
- Develop IDE support (LSP)

Phase 4: Backend & Runtime

- Integrate LLVM/MLIR backend

- Implement work-stealing scheduler
- Add GPU kernel generation (CUDA/ROCm)
- Build memory management system

Phase 5: Standard Library

- Implement core algorithms
- Create ML module (convolution, etc.)
- Build graphics module
- Add linear algebra operations

Phase 6: Optimization & Testing

- Profile and optimize performance
- Benchmark against existing solutions
- Build comprehensive test suite
- Write documentation and tutorials

Phase 7: Ecosystem

- Package manager
- Build system
- Community tools and libraries
- Production deployments

13. Technical Foundation

Fourier's implementation will leverage state-of-the-art compiler technology and recent research in parallel computing, machine learning compilers, and GPU programming.

13.1 Compiler Infrastructure

MLIR (Multi-Level Intermediate Representation)

Modern compiler infrastructure designed for heterogeneous computing, used by TensorFlow, PyTorch, and other ML frameworks

LLVM Backend

Proven code generation and optimization infrastructure with support for multiple architectures

Polyhedral Compilation

Advanced loop optimization and automatic parallelization techniques

13.2 Research Foundations

The language design draws from recent research in several areas:

- **Automatic Differentiation:** Papers on differentiable programming and gradient computation
- **GPU Compilation:** Triton (OpenAI), Halide, and modern GPU programming models
- **Type Systems:** Linear types, affine types, and ownership for safe parallelism
- **Effect Systems:** Tracking side effects for safe parallelization
- **Work-Stealing:** Modern task-based parallel runtime systems

14. Conclusion

Fourier represents a new approach to programming for compute-intensive workloads. By making parallelization and GPU acceleration the default rather than an afterthought, it aims to make high-performance computing accessible to a broader audience.

The language is designed to bridge the gap between productivity and performance, allowing developers to write clear, readable code that automatically leverages modern hardware. Whether training neural networks or rendering real-time graphics, Fourier aims to be the natural choice for developers who need both ease of use and maximum performance.

This specification represents version 0.1 and will evolve as the language develops. Feedback, contributions, and community involvement are encouraged as we work together to build a language for the next generation of compute-intensive applications.